AMERICAN LATIN CLASS ATTENDANCE SYSTEM

# Python Analytics Api Evidence

**Table of Contents**



# Python Analytics API Evidence

Date: 2026-07-03

Public base URL:

```
https://18-217-255-109.sslip.io/api/analytics/v1
```

Postman collection:

```
postman/American-Latin-Class-Analytics-API.postman_collection.json
```

Environment:

```
postman/American-Latin-Class.postman_environment.json
```

## Objective

This evidence proves the Python API is a real project API, not only an authentication check. The API is implemented with FastAPI and provides academic analytics over the American Latin Class PostgreSQL database.

The API calculates:

- Student attendance risk.
- Student scholarship readiness.
- Branch performance summary.
- Teacher workload and estimated payment.

The same JWT session token created by the Node Auth API is used to protect the analytics endpoints.

## Evidence Summary

| Evidence | Method | URI | Expected Status | |
|---|---|---|---|---|
| Health check | `GET` | `/health` | `200` | |
| No-token protection | `GET` | `/students/{{student_id}}/attendance-risk` | `401` | |
| Attendance risk | `GET` | `/students/{{student_id}}/attendance-risk` | `200` | |
| Scholarship readiness | `GET` | `/students/{{student_id}}/scholarship-readiness` | `200` | |
| Branch performance | `GET` | `/branches/{{branch_id}}/performance-summary` | `200` | |

| Evidence | Method | URI | Expected Status | |
|---|---|---|---|---|
| Teacher workload | `GET` | `/teachers/{{teacher_id}}/workload-summary` | `200` | |
| Logout validation | `POST` + `GET` | `/api/v1/auth/logout`, then analytics route | `200`, then `401` | |

## 1. Health Check

Postman request:

```
GET {{analytics_base_url}}/health
```

Expected response:

```
{
  "success": true,
  "message": "Analytics API healthy",
  "data": {
    "service": "American Latin Class Analytics API",
    "status": "healthy",
    "authRequired": true
  }
}
```

What this proves:

- The Python API is deployed.
- FastAPI is running behind Nginx.
- Authentication is enabled for protected endpoints.

## 2. Student Attendance Risk

Postman request:

```
GET {{analytics_base_url}}/students/{{student_id}}/attendance-risk
Authorization: Bearer {{session_token}}
```

Actual verified data:

| Field | Value |
|---|---|
| Student | `Camila Rojas REAL-20260624154645` |
| Level | `B1` |
| Total records | `6` |
| Attended records | `6` |
| Absent records | `0` |
| Late records | `1` |
| Attendance rate | `100.0` |
| Risk level | `low` |

Expected response excerpt:

```
{
  "success": true,
  "message": "Student attendance risk",
  "data": {
    "studentName": "Camila Rojas REAL-20260624154645",
    "totalRecords": 6,
    "attendedRecords": 6,
    "attendanceRate": 100.0,
    "riskLevel": "low",
    "recommendation": "Student attendance is healthy."
  }
}
```

How the API works:

```
attendanceRate = attendedRecords / totalRecords * 100
attendanceRate = 6 / 6 * 100 = 100.0
```

Risk rule:

| Condition | Risk |
|---|---|
| No records | `no_data` |
| Attendance below 75% | `high` |
| Attendance below 90% | `medium` |
| Attendance 90% or higher | `low` |

Because the student has `100.0%`, the API returns `riskLevel: low`.

## 3. Student Scholarship Readiness

Postman request:

```
GET {{analytics_base_url}}/students/{{student_id}}/scholarship-readiness
Authorization: Bearer {{session_token}}
```

Actual verified data:

| Field | Value |
|---|---|
| Attendance rate | `100.0` |
| Required attendance rate | `90.0` |
| Period months | `2` |
| Scholarship eligible | `true` |
| Missing percentage points | `0` |

Expected response excerpt:


```
{
  "success": true,
  "message": "Student scholarship readiness",
  "data": {
    "attendanceRate": 100.0,
    "requiredAttendanceRate": 90.0,
    "periodMonths": 2,
    "scholarshipEligible": true,
    "missingPercentagePoints": 0,
    "recommendation": "Student meets the scholarship attendance
        threshold."
  }
}
```


How the API works:

```
scholarshipEligible = attendanceRate >= requiredAttendanceRate
scholarshipEligible = 100.0 >= 90.0 = true
```

This endpoint proves the API reads the active scholarship rule from the database and applies it to the selected student.

## 4. Branch Performance Summary

Postman request:

```
GET {{analytics_base_url}}/branches/{{branch_id}}/performance-summary
Authorization: Bearer {{session_token}}
```

Actual verified data:

| Field | Value |
|---|---|
| Branch | `Santo Domingo Central REAL-20260624154645` |
| City | `Santo Domingo` |
| Students total | `6` |
| Students active | `6` |
| Teachers total | `3` |
| Class groups total | `2` |
| Class sessions total | `6` |
| Attendance records total | `10` |
| Attendance rate | `90.0` |
| Pending enrollment requests | `5` |
| Performance level | `strong` |

Expected response excerpt:


```
{
  "success": true,
  "message": "Branch performance summary",
  "data": {
    "branchName": "Santo Domingo Central REAL-20260624154645",
    "studentsTotal": 6,
    "teachersTotal": 3,
    "classGroupsTotal": 2,
    "classSessionsTotal": 6,
    "attendanceRecordsTotal": 10,
    "attendanceRate": 90.0,
    "pendingEnrollmentRequests": 5,
    "performanceLevel": "strong"
  }
}
```


How the API works:

- Counts students assigned to the branch.
- Counts active students.
- Counts teachers assigned to the branch.
- Counts class groups and class sessions.
- Aggregates attendance records for branch students.
- Calculates an attendance rate and classifies branch performance.

Performance rule:

| Condition | Performance |
|---|---|
| No attendance records | `no_data` |
| Attendance 90% or higher | `strong` |
| Attendance 75% to 89.99% | `watch` |
| Attendance below 75% | `needs_attention` |

Because this branch has `90.0%`, the API returns `performanceLevel: strong`.

## 5. Teacher Workload Summary

Postman request:

```
GET {{analytics_base_url}}/teachers/{{teacher_id}}/workload-summary
Authorization: Bearer {{session_token}}
```

Actual verified data:

| Field | Value |
|---|---|
| Teacher | `Isabella Torres REAL-20260624154645` |
| Hourly rate | `22.5` |
| Check-ins total | `1` |
| Completed check-ins | `1` |
| Open check-ins | `0` |
| Class sessions total | `1` |
| Completed hours | `3.0` |
| Estimated pay | `67.5` |

Expected response excerpt:

```
{
  "success": true,
  "message": "Teacher workload summary",
  "data": {
    "teacherName": "Isabella Torres REAL-20260624154645",
    "hourlyRate": 22.5,
    "completedHours": 3.0,
    "estimatedPay": 67.5
  }
}
```


How the API works:

```
estimatedPay = completedHours * hourlyRate
estimatedPay = 3.0 * 22.5 = 67.5
```

This proves the Python API calculates teacher payment projections from database attendance records.

## 6. Recommended Postman Screenshots

Capture these screens for the final evidence package:

| Screenshot | What To Show |
|---|---|
| 1 | `GET /api/analytics/v1/health` with `200 OK`. |
| 2 | `GET /students/{{student_id}}/attendance-risk` without token returning `401`. |
| 3 | `POST /api/v1/auth/login` returning `200 OK` and session token. |
| 4 | `GET /students/{{student_id}}/attendance-risk` with Bearer token returning calculated risk. |
| 5 | `GET /students/{{student_id}}/scholarship-readiness` with scholarship eligibility. |
| 6 | `GET /branches/{{branch_id}}/performance-summary` with branch totals. |
| 7 | `GET /teachers/{{teacher_id}}/workload-summary` with estimated pay. |
| 8 | `POST /api/v1/auth/logout` returning `200 OK`. |
| 9 | Repeat a protected analytics endpoint after logout returning `401`. |

## Conclusion

This evidence demonstrates that the Python API:

- Is deployed and reachable through the public HTTPS domain.
- Uses FastAPI as a separate backend API.
- Reads real PostgreSQL/RDS records.
- Performs domain-specific calculations.
- Protects private analytics with the same JWT session system as the Node backend.
- Rejects revoked tokens after logout.